

El Gato | Tic Tac Toe - Enunciado

// ⚙ Stack técnico requerido

Nota: Esta sección no explica las tecnologías — ya las conoces. Es un recordatorio explícito de qué se espera de cada una **en este proyecto**. Todo lo listado aquí es obligatorio y evaluado.

LOMBOK

- `@RequiredArgsConstructor` en controllers y services — sin `@Autowired`
- `@Builder` en todos los DTOs y entidades
- `@Slf4j` en cada clase que loguea — sin instanciar `LoggerFactory` a mano
- `@Data` sólo donde corresponde. En entidades preferir `@Getter` + `@Setter` explícito

LOGGING – Logback + SLF4J

- `log.info` — flujo normal: partida creada, movimiento registrado, jugador unido
- `log.warn` — situación recuperable: fallback activado, timeout detectado
- `log.error` — fallo real: excepción inesperada, error de base de datos
- Cero `System.out.println` en todo el proyecto
- Mensaje con contexto:


```
log.info("Movimiento registrado:
game={} player={} pos={}", ...)
```

SPRING SECURITY – Basic Auth (ambos MS, roles distintos)

MS CLIENTE – CREDENCIALES DE JUGADOR

- Basic Auth con `username` + `password` del jugador registrado
- El jugador autenticado se obtiene del `SecurityContext` — sin `X-Player-Id` manual
- Password hashado con `BCryptPasswordEncoder` — nunca texto plano en DB
- `POST /api/players` (registro) es el único endpoint público

MS SERVER – CREDENCIALES DE MICROSERVICIO

- Basic Auth con credenciales de MS, no de jugador
- Sólo los MS Clientes registrados pueden llamar al server
- El server expone `POST /api/clients/register` como endpoint público para el auto-registro
- Todos los demás endpoints del server requieren autenticación de MS

- Todo lo demás requiere autenticación de jugador

- Credenciales del cliente en `application.yml` via variables de entorno: `${CLIENT_ID}` / `${CLIENT_SECRET}`

AUTO-REGISTRO AL ARRANQUE – APPLICATIONRUNNER EN EL MS CLIENTE

- Al iniciar, el MS Cliente ejecuta un `ApplicationRunner` (o `CommandLineRunner`) que llama a `POST /api/clients/register` en el server
- Envía: `clientId`, `clientSecret` (hasheado en server), `baseUrl` del cliente
- Si el cliente ya está registrado (`clientId` existe), el server responde `200 OK` sin error
- Si el server no está disponible al arranque, el cliente logea un `warn` y continúa — el registro se puede reintentar
- Una vez registrado, todas las llamadas del cliente al server incluyen el header `Authorization: Basic ...` con las credenciales del MS

DTOS POR CAPAS

- `*Request` / `*Response` — en la capa controller (entrada y salida HTTP)
- `*Command` / `*Result` — entre controller y service
- Las `*Entity` nunca salen del service hacia el controller
- Sin clases llamadas `*Model`, `*Domain`, `*VO` por ahora
- Todos los DTOs con `@Builder` y validaciones `javax.validation` donde corresponde

BEAN VALIDATION

- `@Valid` en todos los parámetros `@RequestBody` de los controllers
- `@NotNull`, `@NotBlank`, `@Min`, `@Max` en los `*Request` que lo requieran
- Errores de validación capturados en `@ControllerAdvice` → retornan `400` con body estructurado
- Sin validar campos a mano en el service cuando Bean Validation lo puede hacer

CONFIGURACIÓN – YAML

- Solo archivos `.yml` — cero `.properties` en el proyecto
- Perfil `dev` separado: `application-dev.yml`
- En `dev`: consola H2/logs en DEBUG habilitados. En producción: deshabilitados.
- Todo valor configurable vía variable de

FLYWAY

- Migraciones en `src/main/resources/db/migration/`
- Nomenclatura: `V1__create_games.sql`, `V2__add_game_type.sql`, etc.
- `spring.jpa.hibernate.ddl-auto: validate` — nunca `create` ni `update` en ningún perfil

- Todo valor configurable via variable de entorno: `${DB_URL}`, `${SERVER_PORT}`, `${TARGET_SERVER_URL}`
- Sin strings hardcoded de URLs, puertos ni credenciales en el código Java

- Cada tabla tiene su migración. Cambios de esquema = nueva migración, no editar versiones anteriores
- Ambos microservicios tienen sus propias migraciones independientes

BASE DE DATOS — MySQL

- MySQL para ambos microservicios — sin H2 en producción
- Credenciales solo en variables de entorno, no en el `yaml` commitado
- Cada microservicio tiene su propia base de datos — nunca comparten esquema
- Nombres de tablas en `snake_case`: `games`, `game_registry`, `players`

DOCKER + DOCKER COMPOSE

- `Dockerfile` multistage en cada MS: stage Maven para compilar, stage JRE slim para correr
- Un solo `docker-compose.yml` en la raíz que levante todo el sistema
- Servicios mínimos en Compose: `gato-server`, `gato-client-1`, `gato-client-2`, `db-server`, `db-client-1`, `db-client-2`
- `gato-client-1` y `gato-client-2` usan la **misma imagen**, distinta configuración via `environment:`
- `depends_on` configurado: los MS esperan a que la DB esté lista
- `docker compose up` desde la raíz levanta todo. Sin pasos manuales adicionales.

JUNIT 5 + MOCKITO

- Tests en `src/test/java`, misma estructura de paquetes que el código fuente
- **Obligatorio en server:** tests unitarios de `GameValidatorService` — un test por cada regla de validación
- **Obligatorio en server:** tests del detector de victoria — las 8 combinaciones ganadoras + empate + tablero sin resultado

OPENAPI — Springdoc

- Dependencia `springdoc-openapi-starter-webmvc-ui` en ambos MS
- `/swagger-ui.html` accesible y funcional en cada servicio
- `@Tag` en cada controller, `@Operation` en cada endpoint
- `@ApiResponse` documentando al menos los códigos de éxito y el error principal por endpoint
- En producción/Railway: Swagger

- **Obligatorio en cliente:** test del fallback — mockear `RestTemplate` lanzando `ResourceAccessException`, verificar respuesta 503 estructurada
- `mvn test` debe pasar sin errores antes de entregar
- Sin `@SpringBootTest` en los tests unitarios — tests de unidad pura con Mockito

puede quedar habilitado para este ejercicio

DEPLOY — Railway / Render

- Al menos el **MS Server** desplegado y con URL pública funcional
- Al menos **una instancia del MS Cliente** desplegada

